

SEBI ICDR Formal Compliance Verification

Weekly Progress — May 23, 2026

Project: Automated formal verification of IPO eligibility under SEBI ICDR 2018

Framework: Lean 4 · LeanDojo · ReProver (Song, Yang & Anandkumar, Caltech)

Regulations covered: Definitions (Reg 2, 68 sub-clauses) + Rules (Regs 4-23, gold standard)

Executive Summary

This week we completed the **definitions extraction layer** — converting Regulation 2's 68 statutory definitions into structured data — and used it to build a **definition-grounded type schema** (`Core.lean`) for the Lean 4 formal library. We confirmed that the proof infrastructure works end-to-end: a real compliance gate over a concrete issuer (`sample_passes_reg5`) was formally verified by the Lean kernel. The next stage is automating this across all 81 gold-standard rules using LeanDojo.

MILESTONE	STATUS
Reg 2 definitions extracted (68 sub-clauses)	✓ Complete
Inductive type constructors added to JSONL	✓ Complete
<code>Core.lean</code> regenerated — definition-grounded	✓ Complete
<code>lake build</code> passes (Lean 4.28.0)	✓ Complete
First real closed proof (<code>sample_passes_reg5</code>)	✓ Complete
LeanCopilot native build (Windows)	⚠ Blocked — linker issue
<code>Reglib_gold</code> + <code>GoldProbe.lean</code>	➡ Next
<code>verify_one.py</code> APOLLO automation	➡ Next

1. Full Pipeline Overview

The system takes two inputs — the SEBI ICDR 2018 PDF and (eventually) a company's IPO filing — and produces formal compliance proofs in Lean 4. Here is the complete data flow:

```

SEBI ICDR 2018 PDF
├── llm_extract_rules.py --mode rules
│   Two-pass LLM extraction (Qwen2.5:14b, local)
│   Output: rules_refactored_v8.jsonl (301 regulations planned)
├── llm_extract_rules.py --mode definitions ← NEW this week
│   Two-pass LLM extraction of Reg 2 sub-clauses
│   Output: definitions_icdr_reg2.jsonl (68 sub-clauses) ✓
└── Both JSONLs
    ├── generate_reglib.py --rules ... --defs ... ← UPDATED this week
    │   Generates the Lean 4 library (Reglib/)
    │   ├── definitions/Core.lean (types, structs)
    │   ├── rules/Chapter2_Part*.lean (rule predicates)
    │   └── rules/Compliance.lean (gates + sample proof) ✓
    └── Reglib/ (lake build ✓)
        ├── LeanCopilot (VS Code) Interactive proof exploration
        └── verify_one.py + LeanDojo Automated batch proof search ← NEXT
  
```

GOLD STANDARD TRACK (PARALLEL)

Rather than waiting for all 301 regulations, we use the verified gold standard (Regs 4-23, 143 rules) as the first complete test bed:

```

gold_standard_regs_4_23.jsonl (143 rules, manually verified)
├── generate_reglib.py --rules gold_standard --defs definitions
│   Output: Reglib_gold/
│   ├── Core.lean (same as Reglib/)
│   ├── Chapter2_Part*.lean
│   ├── Compliance.lean
│   └── GoldProbe.lean (81 sorry theorems → LeanCopilot targets)
  
```

2. Definitions Extraction

WHAT IS REGULATION 2 AND WHY DOES IT MATTER?

Regulation 2 of SEBI ICDR 2018 is the **statutory definitions chapter**. It defines 68 terms — from "advertisement" to "working day" — that every other regulation refers to. Before this week, every type in our Lean schema was *inferred* from how rules use these terms (e.g., because Reg 5 checks `is_debarred`, we guessed it was a yes/no field). Now the types come directly from what the statute says the terms *mean*.

THE TWO-PASS EXTRACTION PROCESS

Pass 1 — Identify sub-clauses. A sliding 2-page window of PDF text is sent to the local LLM. It returns a list of sub-clause identifiers: `(a) "Act" means...`, `(b) "advertisement" includes...`, and so on.

Pass 2 — Structure each definition. For each identified sub-clause, a second targeted LLM call extracts: - `term`: the defined term (e.g., "anchor investor") - `lean_type_hint`: what kind of Lean type this should become - `maps_to_field`: the field name on the `Issuer` struct - `cross_references`: other regulations referenced in the definition

Post-processing — Extract constructors. For terms that represent enumerated categories (like "qualified institutional buyer" which lists 14 specific categories), a third LLM pass reads the definition text and extracts the list of named variants.

RESULTS

- **65 sub-clauses extracted automatically**, confidence ≈ 1.0 , zero false positives
- **3 manually patched**: sub-clauses `(oa)`, `(bbbb)`, and `(eeea)` were inserted by 2023/2025 amendments mid-sequence — the extractor missed them because they break alphabetical order
- **Final total: 68 definitions**, covering sub-clauses `(a)` through `(mmm)`

Key definitions and their Lean roles:

REG 2	TERM	TYPE ROLE IN LEAN
(aa)	issuer	Root of the <code>Issuer</code> struct
(oo)	promoter	<code>Promoter</code> struct — 3 statutory tests as Bool fields
(ss)	qualified institutional buyer	<code>QualifiedInstitutionalBuyer</code> inductive — 14 constructors
(eee)	specified securities	<code>SpecifiedSecurities</code> inductive — 3 constructors
(hh)	net worth	<code>INR_Crore</code> alias (financial threshold)
(lll)	wilful defaulter / fraudulent borrower	Bool flag on <code>Issuer</code> and <code>Promoter</code>
(w)(q)(h)(xx) (nn)(tt)	issue types	<code>IssueType</code> inductive — 6 constructors

3. What Are Constructors and Why Do We Need Them?

This section explains the concept from first principles for readers unfamiliar with formal verification.

THE PROBLEM: CATEGORIES NEED TO BE EXHAUSTIVE AND PRECISE

In compliance law, many terms are *categorical* — a thing belongs to one of a fixed list of named types. "Qualified institutional buyer" is not a number or a free-text description; it is a membership category. SEBI ICDR Reg 2(ss) lists exactly 14 types of entity that qualify: mutual funds, venture capital funds, scheduled commercial banks, and so on. A company applying for an IPO is or is not a QIB, and if it is, it is specifically one of those 14 types.

A proof system needs this to be encoded precisely. If we wrote `qib_type : String` and stored `"mutualFund"` as text, Lean cannot guarantee that a rule about QIBs is being applied to a real QIB category — someone could write `"mutalFund"` (typo) and the proof system would accept it. The entire point of formal verification is to rule out this class of error.

THE SOLUTION: INDUCTIVE TYPES

An **inductive type** is a type whose complete set of possible values is declared upfront and named. Think of it as a Python `Enum` but with formal guarantees. In Lean:

```
-- Reg 2(ss): qualified institutional buyer
-- Exactly 14 possible values, derived from the statute
inductive QualifiedInstitutionalBuyer where
  | mutualFund
  | ventureCapitalFund
  | alternativeInvestmentFund
  | foreignVentureCapitalInvestor
  | foreignPortfolioInvestor
  | publicFinancialInstitution
  | scheduledCommercialBank
  | multilateralDFI
  | stateIndustrialDevCorp
  | insuranceCompany
  | providentFund          -- min corpus ₹25 crore
  | pensionFund           -- min corpus ₹25 crore, PFRDA registered
  | nationalInvestmentFund
  | departmentOfPostsInsuranceFund
  | systematicallyImportantNBFC
deriving Repr, DecidableEq
```

Each named variant (`mutualFund`, `scheduledCommercialBank`, etc.) is a **constructor** — a way to create a value of this type. `deriving DecidableEq` tells Lean's proof engine that it can compare two `QualifiedInstitutionalBuyer` values and decide if they are equal, which is necessary for automated proof search.

WHY WE EXTRACT CONSTRUCTORS FROM THE STATUTE — NOT FROM OUR OWN JUDGMENT

The specific constructors are not invented by us. They are extracted directly from the statutory text of Regulation 2 by the LLM post-processing pass described in Section 2. Reg 2(ss) lists "(i) a mutual fund... (ii) foreign portfolio investor... (iii) a public financial institution..." — those become the constructors. This is the key guarantee: the formal types are *legally faithful* to the statute.

THE THREE INDUCTIVE TYPES USED IN REGLIB

`SpecifiedSecurities` — Reg 2(eee)

"means equity shares and convertible securities"

Plus the inserted amendment Reg 2(eeee) for SR equity shares (superior voting rights):

```
inductive SpecifiedSecurities where
  | equityShares
  | convertibleSecurities
  | srEquityShares      -- Reg 2(eeee): superior voting rights
deriving Repr, DecidableEq
```

IssueType — combines Reg 2(w)(q)(h)(xx)(nn)(tt)

These six regulations each define a distinct type of securities offering. We combine them into one inductive because rules throughout ICDR are conditioned on issue type — Reg 5 (IPO eligibility) applies only to `initialPublicOffer`, not to `rightsIssue`.

```
inductive IssueType where
  | initialPublicOffer  -- Reg 2(w): unlisted issuer to public
  | furtherPublicOffer  -- Reg 2(q): listed issuer to public
  | compositeIssue      -- Reg 2(h): public-cum-rights simultaneously
  | rightsIssue         -- Reg 2(xx): to shareholders on record date
  | preferentialIssue   -- Reg 2(nn): private placement to select persons
  | qualifiedPlacement   -- Reg 2(tt): private placement to QIBs
deriving Repr, DecidableEq
```

4. The Core.lean Schema — Three Key Structures

`Core.lean` defines the data types that represent a company seeking IPO approval. Think of it as the **data model** — the structured representation of everything Lean needs to know about a company to check whether it meets ICDR requirements.

Before this week, every type in this model was reverse-engineered from how rules use terms. Now the types are grounded in Regulation 2's statutory definitions. The difference matters for proof automation: a type grounded in `INR_Crore` gives the proof engine semantic information (this is a rupee amount) that `Nat` alone does not.

STRUCTURE 1: PROMOTER

A promoter is a person or entity that controls or has founded the company applying for IPO approval. Regulation 2(oo) defines exactly three ways a person can be a promoter — these become three Bool fields:

```

-- Reg 2(ii): A person included as promoter under any of three limbs
structure Promoter where
  -- Limb (i): named in the offer document or annual return (s.92 Companies Act)
  named_in_offer_doc      : Bool
  -- Limb (ii): holds control as shareholder, director, or otherwise
  has_control             : Bool
  -- Limb (iii): board of directors is accustomed to act on their advice
  board_acts_on_instructions : Bool
  -- Proviso: if acting purely in professional capacity, excluded from (iii)
  acting_only_professionally : Bool
  -- Shareholding percentage
  holding_pct             : Pct          -- Pct = Nat (0-100)
  -- Disqualification flags (Regs 5, 2(l), 2(p))
  is_debarred             : Bool
  is_wilful_defaulter_or_fraudulent : Bool
  is_fugitive_economic_offender : Bool

```

The `Issuer` struct holds a `List Promoter` — a company can have multiple promoters, and rules like Reg 5 must be satisfied by *all* of them.

STRUCTURE 2: ISSUER

This is the main record — 61 fields representing the complete compliance profile of a company. Key fields:

```

structure Issuer where
  -- Financial thresholds (three-year history as a list)
  net_tangible_assets_3yr : List INR_Crore -- Reg 6(1)(a): must be ≥ ₹3cr each year
  net_worth_3yr           : List INR_Crore -- Reg 6(1)(c): must be ≥ ₹1cr each year
  operating_profit_last_3yr : List INR_Crore

  -- Disqualification flags (must be false to pass Reg 5)
  is_debarred             : Bool
  is_wilful_defaulter_or_fraudulent_borrower : Bool
  is_fugitive_economic_offender : Bool

  -- Structural configuration (from inductive types)
  issue_type              : IssueType      -- which kind of offering
  specified_securities_type : SpecifiedSecurities -- what is being issued

  -- Promoter list (rules apply to ALL promoters)
  promoters : List Promoter

  -- ... 50 further fields for lock-in periods, contribution %,
  -- debenture trustee, depository agreements, etc.

```

`INR_Crore` is a type alias for `Nat` (natural number) with the semantic annotation that the unit is crore rupees. This is important for proof readability:

`net_tangible_assets_3yr : List INR_Crore` immediately tells a reader the financial unit, not just that it is a list of numbers.

STRUCTURE 3: `SAMPLE_COMPLIANT_ISSUER`

This is not a type definition — it is a *concrete instance* of `Issuer`: a specific company (fictional) with values chosen to satisfy all ICDR conditions. Think of it as a unit test fixture or a reference data point. It serves two purposes:

1. **Smoke test:** if `lake build` compiles without errors, the schema is internally consistent
2. **Proof target:** the theorem `sample_passes_reg5` formally proves that this specific issuer satisfies Regulation 5

Here is the actual `sample_compliant_issuer` from the generated `Compliance.lean`:

```
def sample_compliant_issuer : Issuer := {
  -- Financial history: three years, all above thresholds
  net_tangible_assets_3yr := [10, 12, 15], -- ₹10cr, ₹12cr, ₹15cr
  net_worth_3yr           := [10, 12, 15],
  operating_profit_last_3yr := [10, 12, 15],

  -- Disqualification: all false → passes Reg 5
  is_debarred                := false,
  is_wilful_defaulter_or_fraudulent_borrower := false,
  is_fugitive_economic_offender := false,

  -- Configuration
  issue_type                := IssueType.initialPublicOffer,
  specified_securities_type := SpecifiedSecurities.equityShares,

  -- One promoter, clean record
  promoters := [{
    named_in_offer_doc      := true,
    has_control              := true,
    board_acts_on_instructions := false,
    acting_only_professionally := false,
    holding_pct              := 25,
    is_debarred              := false,
    is_wilful_defaulter_or_fraudulent := false,
    is_fugitive_economic_offender := false
  }],

  -- ... 50+ further fields set to compliant values
}
```


5. The First Formal Proof — `sample_passes_reg5`

WHAT REGULATION 5 SAYS

Regulation 5 of SEBI ICDR 2018 sets the **disqualification conditions** for IPO eligibility. An issuer is ineligible if:

- **(a)** The issuer, any promoter, promoter group member, director, or selling shareholder is debarred from accessing capital markets by SEBI
- **(b)** Any promoter or director of the issuer is a promoter or director of another company currently debarred by SEBI
- **(c)** The issuer or any of the above persons is categorised as a wilful defaulter or fraudulent borrower
- **(d)** Any of the above persons is a fugitive economic offender under the Fugitive Economic Offenders Act 2018

In Lean, each sub-clause becomes a predicate (a function that takes an `Issuer` and returns a proposition):

```
def reg_5_1_a (issuer : Issuer) : Prop :=
  issuer.is_debarred = false

def reg_5_1_b (issuer : Issuer) : Prop :=
  issuer.has_debarred_promoter_or_director = false

def reg_5_1_c (issuer : Issuer) : Prop :=
  issuer.is_wilful_defaulter_or_fraudulent_borrower = false

def reg_5_1_d (issuer : Issuer) : Prop :=
  issuer.is_fugitive_economic_offender = false

-- The composite gate: all four must hold
def reg_5_eligible (issuer : Issuer) : Prop :=
  reg_5_1_a issuer ∧ reg_5_1_b issuer ∧ reg_5_1_c issuer ∧ reg_5_1_d issuer
```

THE PROOF — LINE BY LINE

The theorem states: "*the sample compliant issuer satisfies Regulation 5.*"

```
theorem sample_passes_reg5 :
  reg_5_eligible sample_compliant_issuer := by
  unfold reg_5_eligible reg_5_1_a reg_5_1_b reg_5_1_c reg_5_1_d
  simp [sample_compliant_issuer, List.all]
```

What each step does:

1. `unfold reg_5_eligible reg_5_1_a reg_5_1_b reg_5_1_c reg_5_1_d`

Lean replaces the function names with their definitions. After this step, the proof goal becomes a concrete conjunction of equality checks — something like

`false = false ∧ false = false ∧ false = false ∧ false = false`.

2. `simp [sample_compliant_issuer, List.all]`

Lean's simplification engine substitutes the actual field values from

`sample_compliant_issuer` (e.g., `is_debarred = false`,

`is_wilful_defaulter_or_fraudulent_borrower = false`) and evaluates. `false = false`

is trivially true, so every conjunct resolves, and the proof closes.

Why this matters: This is not a test that *runs* and prints "PASS". It is a mathematical proof that the Lean kernel has *verified*. The kernel is a small, auditable component (~10,000 lines of code) that only accepts proofs it can check by first principles. There is no way to fool it with a plausible-sounding answer — either the proof term is valid or `lake build` fails.

THE FILES INVOLVED

```

Reglib/
├─ ICDR/
│  ├─ definitions/
│  │  └─ Core.lean          ← Issuer, Promoter, IssueType, etc.
│  └─ rules/
│     ├─ Chapter2_Part1.lean ← reg_5_1_a, reg_5_1_b, reg_5_1_c,
│     │                               reg_5_1_d, reg_5_eligible defined here
│     └─ Compliance.lean     ← sample_compliant_issuer defined here
│                               sample_passes_reg5 proved here ✓
├─ lakefile.lean             ← LeanCopilot dependency declared
└─ lean-toolchain             ← Lean 4.28.0

```

6. LeanCopilot — Design and Current Status

WHAT LEANCOPILOT DOES

LeanCopilot (Song, Yang & Anandkumar, 2024 — [lean-dojo/LeanCopilot](https://lean-dojo.com/LeanCopilot)) is a VS Code extension that integrates a neural tactic model into the Lean editor. When a proof is incomplete (has a `sorry` stub), the user places their cursor on the `sorry` and invokes one of two commands:

- `suggest_tactics` — shows a ranked list of candidate next-tactic suggestions. The user picks one, the proof state updates, and the process repeats until the proof closes.
- `search_proof` — runs beam search automatically, trying combinations of tactics until the whole proof closes or the search budget is exhausted.

The underlying model is **ReProver** — a ByteT5 encoder-decoder fine-tuned on Lean 4 proofs from the Mathlib library. It runs entirely locally (~600 MB, downloaded once). No internet connection is used during proof search.

THE PROBE FILE DESIGN

`CopilotProbe.lean` contains three theorems designed to test the workflow:

```
-- Theorem 1: Baseline (already closed – no LeanCopilot needed)
-- Proves that the sample issuer passes Reg 5 using simp.
-- This is the reference proof, identical to sample_passes_reg5 in Compliance.lean.
theorem sample_passes_reg5_copilot_check :
  reg_5_eligible sample_compliant_issuer := by
  unfold reg_5_eligible reg_5_1_a reg_5_1_b reg_5_1_c reg_5_1_d
  simp [sample_compliant_issuer]

-- Theorem 2: LeanCopilot target – sub-clause level
-- suggest_tactics will propose: simp [sample_compliant_issuer]
-- Expected: closes in one tactic suggestion
theorem copilot_reg5_1_a :
  reg_5_1_a sample_compliant_issuer := by
  suggest_tactics -- ← cursor here in VS Code

-- Theorem 3: LeanCopilot target – composite gate
-- search_proof attempts the full proof automatically
-- Expected: closes with the same unfold + simp pattern as Theorem 1
theorem copilot_reg5_eligible :
  reg_5_eligible sample_compliant_issuer := by
  search_proof -- ← cursor here in VS Code
```

CURRENT STATUS ON WINDOWS

`lake build CopilotProbe` did not complete. The LeanCopilot native build fails at the FFI linker step on Windows (`undefined symbol: ctranslate2::destroy_context`), caused by two compounding issues: OneDrive path spaces breaking the `cmd /c copy` commands in the upstream build script, and a stale MSYS2 mirror URL in the LeanCopilot repository. Workaround: clone the project to a short path without spaces (e.g., `C:\compliance`) and rebuild, or use WSL.

What we do have: The proof infrastructure is confirmed working.

`sample_passes_reg5` in `Compliance.lean` is a real closed proof, compiled by `lake build` and accepted by the Lean kernel. The LeanCopilot probe is designed to automate the same pattern across many theorems — once the native build is fixed, no architectural changes are needed.

7. Reglib_gold — The Test Bed

WHY NOT WAIT FOR ALL 301 REGULATIONS?

The full ICDR extraction covers 475 pages and 301 regulations. That run is ongoing. `Reglib_gold` uses the 143 gold-standard rules (Regs 4-23, manually verified for correctness) to validate the entire proof pipeline independently of the full corpus. The architecture is identical — only the rule count changes when the full run completes.

COMPOSITION

COMPONENT	CONTENT	STATUS
<code>Core.lean</code>	Same schema as main Reglib	✓ Ready to generate
<code>Chapter2_Part*.lean</code>	143 gold rules, Regs 4-23	✓ Ready to generate
<code>Compliance.lean</code>	<code>sample_compliant_issuer</code> + <code>sample_passes_reg5</code>	✓ Ready to generate
<code>GoldProbe.lean</code>	81 LeanCopilot targets + 49 trivial provisos	➡ SOON Next

THE 81 PROOF TARGETS

Of the 143 gold rules: - **81 rules** generate real Bool/Nat predicates (LeanCopilot targets — can be proved automatically) - **49 rules** are provisos and explanations (proved by `trivial` — no tactic search needed) - **13 rules** have no `maps_to` field (skipped — rule body is `sorry`)

The 81 real-predicate theorems are the meaningful test. If automated proof search closes $\geq 60\%$, that is the first experimental result of the paper.

8. The Automation Plan — APOLLO Loop

Once tactic patterns are discovered via interactive LeanCopilot, `verify_one.py` automates proof search across all 81 `GoldProbe` theorems using **LeanDojo** — a Python library from the same research group that provides programmatic access to the Lean kernel. No VS Code is `involved`.

```
for each sorry theorem in GoldProbe.lean:
  |
  |— LeanDojo opens Lean environment, loads Reglib_gold
  |— Lean elaborates the theorem → proof state
  |   e.g.: ⊢ sample_compliant_issuer.is_debarred = false
  |
  |— Prompt builder packages:
  |   · proof state
  |   · rule text (from gold rules JSONL)
  |   · field definition (from definitions JSONL) ← why definitions matter
  |
  |— Qwen2.5:14b (local) suggests tactic
  |   e.g.: "simp [sample_compliant_issuer]"
  |
  |— LeanDojo tries tactic against Lean kernel
  |   |— ProofFinished → save to proof_results.jsonl ✓
  |   |— Error/partial → update prompt, retry (max 5 rounds)
  |
  |— After 5 rounds → log as unproved ✗
```

Output: proof_results.jsonl

- Which theorems closed automatically
- How many rounds each took
- Which tactic closed it
- Paper metric: proof success rate over 81 targets

The definitions JSONL plays a key role here: when the LLM sees the proof goal `issuer.net_tangible_assets_3yr.all (· ≥ 3)` and also receives the definition "net tangible assets means the sum of all net assets excluding intangible assets as per AS26/Ind AS 38", it can reason about what arithmetic property to prove, not just pattern-match on the field name.

9. Files Produced This Week

FILE	PURPOSE	STATUS
<code>data/processed/definitions_icdr_reg2.jsonl</code>	68 Reg 2 definitions with Lean types and constructors	✓ Done
<code>scripts/patch_definitions.py</code>	Adds 3 missing amendment sub-clauses (oa, bbbb, eeea)	✓ Done
<code>scripts/add_inductive_constructors.py</code>	LLM post-processing: constructors field for Inductive types	✓ Done
<code>scripts/generate_reglib.py</code>	Updated: <code>--defs</code> flag, data-driven inductives, grounded types	✓ Done
<code>Reglib/ICDR/definitions/Core.lean</code>	61-field Issuer, grounded Promoter, IssueType inductive	✓ Done
<code>Reglib/ICDR/rules/Compliance.lean</code>	sample_compliant_issuer + sample_passes_reg5 (closed)	✓ Done
<code>Reglib/ICDR/CopilotProbe.lean</code>	LeanCopilot probe design (3 theorems)	✓ Done (build pending)
<code>Reglib/lakefile.lean</code>	LeanCopilot dependency, Lean 4.28.0, Lake 5 syntax	✓ Done
<code>scripts/generate_gold_probe.py</code>	Generates GoldProbe.lean with 81 LC targets	→ SOON Next
<code>Reglib_gold/</code>	Full Lean library from gold standard rules	→ SOON Next
<code>scripts/verify_one.py</code>	APOLLO loop: LeanDojo + Qwen, results logging	→ SOON Next

10. Immediate Next Steps

1. **Fix LeanCopilot Windows build** — clone to `C:\compliance`, rebuild. ~1 hr.
 2. **Build Reglib_gold** — `generate_reglib.py` with gold standard + definitions JSONL. ~30 min.
 3. **Generate GoldProbe.lean** — 81 sorry theorems, one per gold rule. ~15 min.
 4. **Interactive LeanCopilot session** — open GoldProbe.lean in VS Code, run `search_proof` on Reg 5 sub-clauses, record which tactic patterns close which rule types. ~1 hr.
 5. **Write verify_one.py** — APOLLO loop skeleton using LeanDojo Python API, Qwen fallback. ~2 hrs.
 6. **Run APOLLO on GoldProbe** — measure proof success rate on 81 theorems. First paper data point. ~2 hrs.
-

Generated May 23, 2026. Lean 4.28.0 · LeanDojo / ReProver · Qwen2.5:14b-instruct (local).